

LockPusher: Object Adaptation with State-Isolated World Models for Pushing under Joint Locks

Anonymous Authors

Abstract—Object adaptation can alter later robot forecasts through feedback in recurrent pushing models. LockPusher addresses this interaction under diagnosed joint locks by retaining a specified robot predictor while adapting object response. Analytic lock projection and separate reference and intervention rollouts preserve each branch’s state and memory. The output combines reference robot forecasts with adapted object forecasts. For deterministic branches with matched initial conditions and actions, robot and pusher forecasts equal the projected reference at every step. Across 27 paired comparisons covering three historical checkpoints, three conditions, and three horizons, selective publication retains the intervention’s 16.74% mean reduction in composite object-state RMSE while preserving reference robot forecasts. Visual replanning completes all 18 core short-distance hardware trials across intact and single-joint-lock conditions.

I. INTRODUCTION

Model-based manipulation uses predicted object motion to select actions before execution. In contact-rich tasks such as pushing, prediction errors can change which action appears most promising [1], [2]. A joint lock further restricts the available motions. Even when the locked joint and its angle are known, the resulting object response remains uncertain. Continuing the task requires a model that incorporates the diagnosed constraint and predicts the interaction outcomes needed to choose a push.

Damage adaptation changes policies or behavior repertoires to recover task performance [3], [4], while learned pushing models estimate contact outcomes from interaction data [2], [5]. ActivePusher connects residual physics with uncertainty-guided data acquisition and planning [6]. We focus on the scope of an object update. Robot forecasts determine predicted pusher motion and can also enter action generation within candidate rollouts. In our historical coupled models, object-state error can decrease while free-state and pusher errors increase, even with zero diagnosed-lock violation (Table III).

Enforcing a known lock and preserving the remaining robot forecast are distinct requirements. We retain a specified robot predictor while adapting the object response. In coupled recurrent models, object corrections can feed back into later robot predictions despite restricting residual outputs to object coordinates. Separate rollout states retain the object update without allowing this feedback to alter the reference prediction.

LockPusher combines analytic lock projection with a state-isolated object correction. A reference predictor, called the carrier, and an intervention branch maintain separate rollout states. LockPusher returns robot coordinates from the carrier and object coordinates from the intervention branch, without

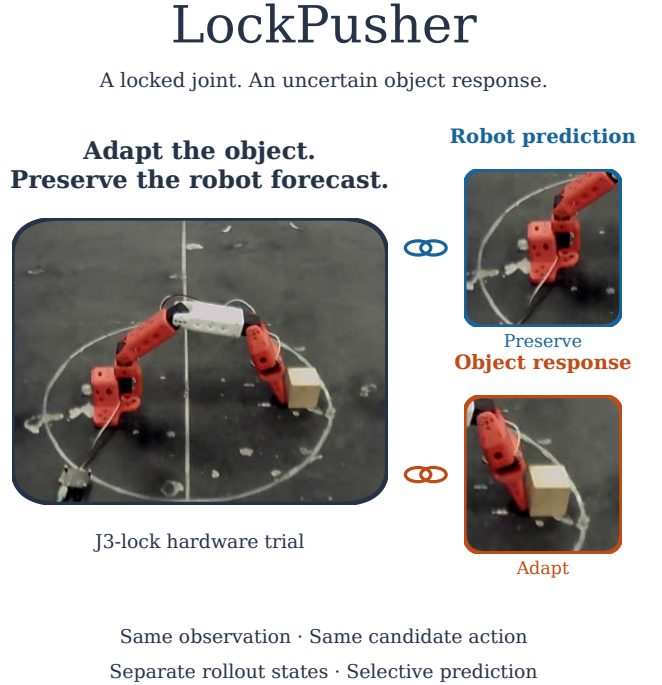


Fig. 1. LockPusher adapts object response under a diagnosed joint lock. The photograph shows an archived j3-lock trial. For the same candidate action, the schematic illustrates preserved robot forecasts and adapted object forecasts; it does not depict measured motion.

feeding this combined output back into either branch. For identical initial observations and action sequences, the construction preserves the projected carrier robot trajectory at every prediction step while allowing the object forecast to change. The intervention branch retains its own robot–object dynamics, so this separation concerns the returned prediction rather than physical independence of the contact dynamics.

The adapted object forecasts rank candidate pushes by predicted terminal distance to the goal. After a portion of the selected action is executed, a fresh observation initializes the next planning cycle.

Historical paired predictions test object gains and robot preservation; a recurrence ablation tests the role of separate physical states. Hardware trials evaluate visual replanning with a robot block that satisfies preservation structurally. Initialization defects leave physical control benefits from the separate large-scale simulations unverified.

Our contributions are:

- A projected, state-isolated world model that permits ob-

ject adaptation while preserving a specified robot forecast for matched inputs and action sequences.

- Paired historical comparisons that separate object-prediction gains from collateral robot-prediction changes, together with a recurrence ablation of reference preservation.
- A visual replanning implementation evaluated in intact and joint-lock hardware trials, with deployment results distinguished from the historical prediction comparisons.

II. RELATED WORK

Adaptation under changed dynamics. Damage recovery can adapt a behavior repertoire [3], condition a policy on inferred extrinsics [7], or tune simulator parameters from observations [8]. These approaches change how a robot acts or how its dynamics are represented. LockPusher studies a different adaptation contract: a diagnosed joint lock is supplied, and a chosen robot predictor is retained while the object response is adapted.

Learning pushing dynamics. Data-efficient pushing models [2] and equivariant architectures [5] use interaction structure to improve motion prediction. Material-adaptive graph models [9] condition dynamics on inferred physical properties. ActivePusher combines residual physics with uncertainty-guided data acquisition and kinodynamic planning [6]. Our method addresses the scope of a learned correction across rollout steps. We study whether object-prediction gains can be retained while preserving a specified robot forecast. LockPusher specifies which state components may change during an adapted rollout.

Structural priors and control. GP pushing models support constrained MPC[2]; SE(2)-equivariant models enforce consistency under planar rigid transformations[5]; PIN-WM identifies 3D rigid-body dynamics from vision for policy transfer[10]. These address control, geometric symmetry, and physical identification; our constraint instead concerns which predicted components may change during adaptation under a known lock.

III. PROBLEM FORMULATION

We study planar pushing with a five-joint arm after a joint lock has been diagnosed. The observed state is $s_t = [r_t, o_t]$, with $r_t = [q_t, \dot{q}_t] \in \mathbb{R}^{10}$ and $o_t = [p_t, v_t] \in \mathbb{R}^4$. Here p_t and v_t denote planar object position and velocity. The simulation object translates with fixed orientation. An action $a_t \in \mathbb{R}^5$ specifies the joint command, and diagnosis $d = (m, \bar{q})$ supplies a binary lock mask and the corresponding angles.

Given an observation, diagnosis, planar goal g , and candidate sequences $a_{0:H-1}^{(k)}$, a world model predicts terminal object positions and selects the candidate with minimum predicted distance to g . We require its returned robot trajectory to equal that of a specified reference predictor for the same observation and candidate. This equality defines the scope of adaptation. The reference is a design input; its preservation and accuracy are evaluated separately. Task performance is measured from executed motion.

IV. METHOD

A. Incorporating the diagnosed lock

LockPusher first incorporates the known constraint into prediction. For each joint j , define

$$\Pi_d(q)_j = (1 - m_j)q_j + m_j\bar{q}_j, \quad (1)$$

$$\Pi_d(\dot{q})_j = (1 - m_j)\dot{q}_j, \quad \Pi_d(a)_j = (1 - m_j)a_j. \quad (2)$$

Object coordinates are unchanged. Applying this projection to model inputs and predicted transitions keeps locked positions fixed and locked velocities zero throughout the rollout. Because m_j is binary, applying the projection twice has the same effect as applying it once. Learned object adaptation therefore operates within an explicitly constrained robot prediction.

B. Object adaptation with separate rollout states

The carrier F_c supplies the reference prediction. An intervention model F_i supplies the adapted object response. Each branch maintains its own physical state s_t^b and recurrent state h_t^b , where $b \in \{c, i\}$. Starting from the same projected observation, both branches receive the same projected candidate action:

$$(s_{t+1}^b, h_{t+1}^b) = F_b(s_t^b, \Pi_d(a_t), d, h_t^b), \quad (3)$$

$$s_{t+1}^b \leftarrow \Pi_d(s_{t+1}^b), \quad (4)$$

$$\hat{s}_{t+1} = [r_{t+1}^c, o_{t+1}^i]. \quad (5)$$

Only the returned state combines the branches. Each branch recurs on its own state, so the intervention retains its learned dynamics without changing the reference recurrence. This distinction matters when corrected object states influence later robot predictions: masking an immediate residual does not remove that later feedback path.

Reference-trajectory preservation. For deterministic branches with matched initial states, carrier parameters, diagnoses, and candidate actions, the returned robot trajectory equals the standalone projected carrier trajectory at every step. The initial carrier states agree. If they agree at step t , the same transition, hidden state, action, and projection produce the same next state. Equation (5) returns that carrier state, establishing the result by induction. The pusher trajectory is also preserved because it is computed from joint positions by the same forward kinematics. For fixed candidates, deterministic evaluation terms computed solely from robot coordinates are also preserved. This property is relative to the chosen carrier; it establishes neither prediction accuracy nor joint dynamical consistency of the returned components.

The robot predictor is a five-node chain graph of width 136. A shared $6 \rightarrow 136 \rightarrow 136$ SiLU encoder embeds joint position, velocity, command, lock mask, lock angle, and chain depth. Two message rounds use a $272 \rightarrow 136 \rightarrow 136$ network and a 136-unit GRU; a second GRU updates temporal memory. A $136 \rightarrow 136 \rightarrow 2$ SiLU head predicts joint increments. The $140 \rightarrow 136 \rightarrow 4$ SiLU object head receives the detached mean robot code and current object state, and predicts object increments. A geometric object residual maps a 12-dimensional

contact descriptor through a 16-unit Tanh hidden layer to four object corrections. The descriptor includes previous and predicted pusher positions, their displacement, pusher-to-object displacement, object velocity, and two contact gates. Each gate is $\sigma((-0.005 - d)/0.002)$, where d is the minimum sampled clearance in meters between two pusher capsules (radii 12 and 8 mm) and a 40-mm axis-aligned box. In this configuration, gates enter the descriptor without additionally multiplying the residual output. In the deployed configuration, the robot block does not consume object state. Separate rollout states also support the historical coupled configuration, in which publication alone must not alter either branch’s recurrence.

C. Matched adaptation

For the large-scale adaptation study, all variants start from the same pretrained carrier. We freeze its robot parameters and update the object head. LockPusher additionally updates its geometric object residual. The no-residual variant excludes this correction, while the global variant learns corrections to all state coordinates. All variants use the same data and optimization budget.

The underlying adaptation model is trained directly. Its predictions $\tilde{s}_t$ minimize normalized squared state errors over an autonomous prediction window; state isolation is applied when returning candidate predictions:

$$\mathcal{L} = \frac{1}{14H} \sum_{t=1}^H \|D^{-1}(\tilde{s}_t - s_t)\|_2^2. \quad (6)$$

The diagonal scale matrix D assigns 0.1 rad to joint positions, 1 rad/s to joint velocities, 0.03 m to object positions, and 0.1 m/s to object velocities. Frozen robot errors contribute no robot-parameter update in the object-only variants. The global residual receives supervision on robot coordinates as well. Training uses ten epochs, with five epochs at horizon 10 followed by five at horizon 25. Each trajectory supplies one randomly positioned contiguous window per epoch. Validation selects the checkpoint with minimum horizon-25 object-position MSE, including the initial checkpoint. These newly adapted weights are evaluated separately from the archived deployment weights.

D. Candidate selection and execution

For each candidate, LockPusher rolls forward and computes $\hat{c}_k = \|\hat{p}_H^{(k)} - g\|_2$. It selects $k^* = \arg \min_k \hat{c}_k$. A new observation reinitializes both branches after execution. Simulation executes action commands; hardware moves toward selected joint references. Their candidate construction and execution intervals are specified below.

V. EXPERIMENTS

Simulation validity. Initial-state audits found tool–object and tool–table penetration and joint-limit violations. Main, calibration, data-budget, and same-weight candidate comparisons therefore lack physical validation. Historical prediction uses a different, physically unchecked reset. Hardware execution is evaluated separately.

Evaluation quantities. Simulation endpoint error is $e_{\text{goal}} = \|p_T - g\|_2$, where p_T is the executed object position and g the assigned goal; it is not prediction error. Goals lie 40–65 or 65–90 mm from the initial object position. All episodes, including failures, enter the mean; success is $e_{\text{goal}} < 30$ mm. Prediction RMSE instead compares predicted and measured coordinates. Hardware uses a separate image-space criterion: five pixels per axis for three consecutive distinct observations, with a 20-pixel core target displacement (approximately 35.1 mm). Its success rate is therefore not directly comparable to the simulation rate.

Matched contrasts with projected nominal dynamics, removal of the object residual, and global adaptation assess archived endpoint differences associated with adaptation, object correction, and correction scope. Historical publication and recurrence comparisons instead fix weights and inputs to test reference preservation.

A. Simulation protocol

The simulated arm pushes a fixed-orientation box with two translational degrees of freedom. We use five single-joint locks and four physical settings: nominal, high damping, weak motor, and mixed perturbations. High damping doubles joint damping; weak motors scale actuator gains to 70% of nominal; the mixed setting applies both. Adaptation, validation, and testing contain 50,000, 2,000, and 6,000 independently reset trajectories, respectively, with disjoint reset seeds. Each trajectory spans 50 steps at 5 ms per step. Each variant uses six fits from the shared pretrained foundation, following the matched adaptation procedure above.

We compare LockPusher with its no-object-residual ablation, global residual adaptation, and projected nominal dynamics. Closed-loop evaluation uses 1,200 shared start–goal problems, comprising 120 problems for each lock and distance band. Each cell contains 30 problems per physical setting, with goal directions sampled uniformly within $\pm 45^\circ$ of the forward axis. At each replan the controller scores 128 candidate commands over a 50-step horizon, executes ten steps, and observes the resulting state. Each episode executes five replans (0.25 s total): each predicts 0.25 s and executes 0.05 s, without stopping on goal entry. Each candidate has five ten-step constant-command segments, sampled independently from $[-0.8, 0.8]$ on free joints and set to zero on the locked joint. Methods use matched problem and candidate-sampling settings, while each controller replans from its own realized state. Endpoint error is Euclidean object-to-goal distance, and success requires error below 30 mm. Position RMSE and velocity RMSE are reported separately in mm and m/s.

B. Archived closed-loop comparison

Table I summarizes the six adaptation fits. LockPusher reduces mean endpoint error from 49.95 to 47.02 mm relative to removing the object residual. The paired improvements range from 1.79 to 5.25 mm across fits, with positive problem-bootstrap intervals in every fit (Fig. 3). Mean success rises from 24.88% to 27.64%. These differences hold in the archived

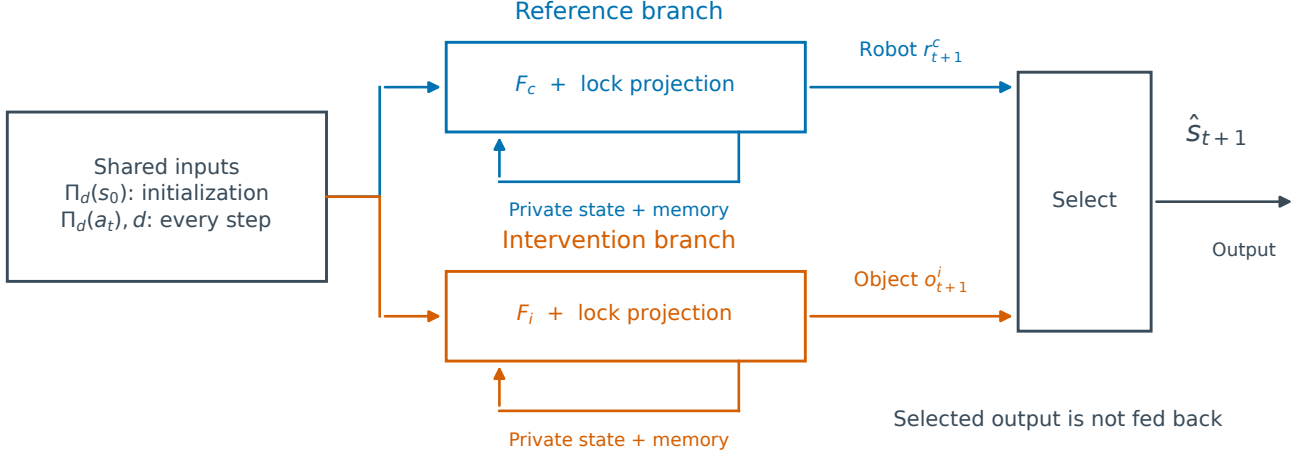


Fig. 2. State-isolated prediction. Both branches retain private rollout states under the same observation, actions, and diagnosed lock. Returned robot coordinates come from the reference; object coordinates come from the intervention. The deployed robot block does not receive object-state input.

TABLE I
FIXED-BUDGET CONTROL (0.25 s) ON 1,200 SHARED PROBLEMS PER FIT.
GOAL DISTANCE INCLUDES FAILURES; SUCCESS REQUIRES DISTANCE
BELOW 30 MM. VALUES ARE MEAN AND SAMPLE SD ACROSS SIX
FIT/CANDIDATE REPEATS.

Method	Goal dist. (mm)	Success (%)
No object residual	49.95 $\pm$ 0.70	24.88 $\pm$ 2.00
Global residual	45.58 $\pm$ 0.75	30.71 $\pm$ 1.47
LockPusher	47.02 $\pm$ 1.21	27.64 $\pm$ 2.21
Projected nominal	54.95 $\pm$ 0.19	19.64 $\pm$ 0.60

simulator outputs; they do not establish improved physical task progress.

The mean initial goal distance is 65.07 mm; LockPusher leaves 47.02 mm, a reduction of 18.05 mm. Success requires ending within a 30 mm goal region; it does not establish precision placement. Mean net object displacement is 60.96 mm, so short execution alone cannot explain the remaining distance. ActivePusher reports mean path-wise SE(2) tracking error in Fig. 7 and uses a 100 $\times$ 100 mm region goal[6]; our terminal goal distance is not directly comparable to that mixed-pose metric.

Relative to projected nominal dynamics, LockPusher reduces mean endpoint error by 7.93 mm and increases mean success by 8.00 percentage points; paired endpoint intervals are positive in all six fits.

Global residual adaptation achieves 45.58 mm mean endpoint error and 30.71% success. Its endpoint error is lower than LockPusher in five of six fits; the paired intervals exclude zero in four. Thus restricting the adapted state retains the prescribed robot prediction but does not minimize endpoint error across all model classes. Open-loop position RMSE likewise does not uniformly improve after object adaptation.

The fixed-checkpoint prediction study is reported in the supplementary analysis, with position and velocity evaluated

separately across all physical conditions. Hardware experiments use the archived deployment weights and are presented in the physical-evaluation section.

C. Validation-selected residual scaling

Candidate random draws vary with the fit seed and are shared across methods within each fit. The stratified bootstrap resamples problems within the ten lock-distance cells, carrying all methods and fits together at each sampled problem index. Intervals are conditional on these fits and their candidate draws.

A separate post-training study selects each fit’s object-residual scale from $\{0, 0.5, 1, 1.5\}$ by H50 position MSE on the existing validation set. On 2,000 fresh shared trajectories, mean per-coordinate position RMSE decreases from 21.845 to 20.750 mm. On 1,200 additional shared closed-loop problems, mean endpoint error decreases from 46.75 to 46.22 mm and success rises from 28.21% to 28.97%. The paired endpoint reduction is 0.532 mm, with a 95% problem-bootstrap interval of [0.357, 0.712] mm conditional on the six fits. Three changed fits have positive intervals, one spans zero, and two retain the original scale. Relative to the no-object-residual model on these fresh problems, calibrated LockPusher reduces endpoint error by 3.398 mm (conditional 95% interval [3.085, 3.710] mm), with positive intervals in all six fits, and raises mean success by 3.90 percentage points. This second problem set shares the initialization defect and cannot independently validate physical benefit. Global adaptation achieves 45.34 mm and 30.60% success on these same problems. Calibration slightly reduces endpoint error, while global adaptation retains a lower mean endpoint error. The calibrated weights were not deployed on hardware.

Full-state versus selective publication. Table III summarizes a separate historical query: checkpoints 27, 37, and 47; high damping, composition, and unseen composition; ten tra-

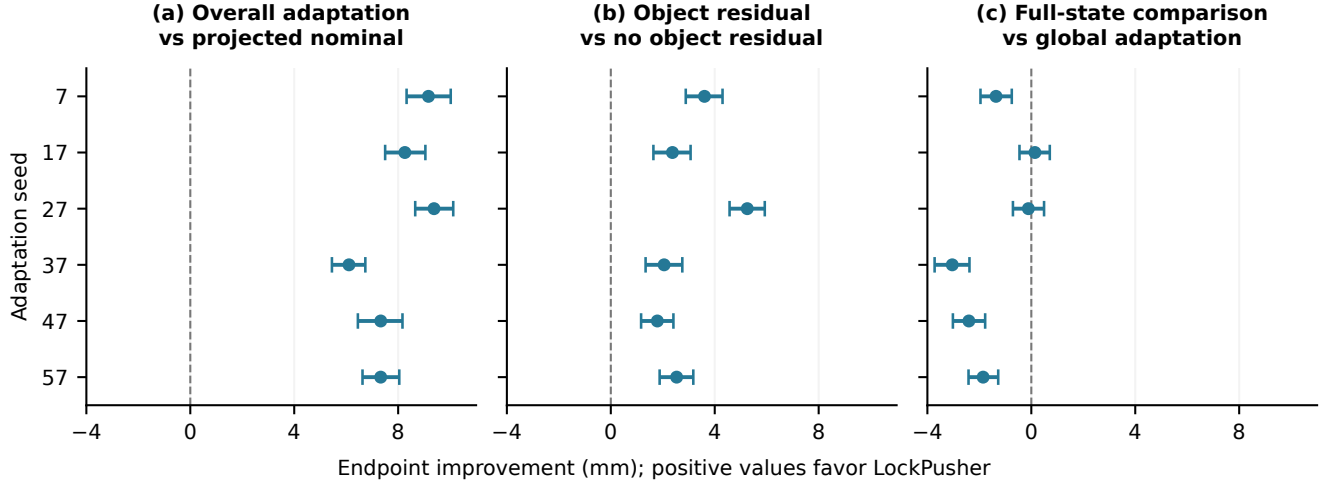


Fig. 3. Paired endpoint comparisons of LockPusher: (a) adaptation relative to projected nominal dynamics; (b) object-residual comparison relative to removing that residual; and (c) comparison with global adaptation. Positive values favor LockPusher. Each point averages 1,200 shared problems for one fit; bars are 95% paired-problem bootstrap intervals (2,000 resamples). All panels share the same scales.

TABLE II

PER-FIT CLOSED-LOOP CHANGES FROM CALIBRATION. POSITIVE VALUES FAVOR CALIBRATION; INTERVALS ARE CONDITIONAL 95% PAIRED-PROBLEM BOOTSTRAP INTERVALS.

Fit	Scale	Error reduction (mm)	Success gain (pp)
7	1	0.00 [0.00, 0.00]	0.00
17	1.5	1.40 [0.91, 1.91]	1.50
27	1.5	0.69 [0.23, 1.16]	1.50
37	1.5	0.98 [0.44, 1.56]	2.50
47	1	0.00 [0.00, 0.00]	0.00
57	1.5	0.11 [-0.42, 0.64]	-0.92

TABLE III

ALL 27 HISTORICAL COMBINATIONS, RELATIVE TO THE CARRIER. OBJECT REDUCTION AVERAGES CHECKPOINT MEANS. MATCH REQUIRES IDENTICAL CARRIER FREE-STATE AND FK-PUSHER SQUARED ERRORS IN EVERY STORED WINDOW.

	Object RMSE reduction (%)	Free worse	Max. free increase (%)	Free/FK match
Carrier	0.00	0/27	0.00	27/27
Full-state	16.74	3/27	6.20	0/27
Selective	16.74	0/27	0.00	27/27

jectories per condition; and horizons 10, 25, and 50. Full-state publication returns intervention robot and object coordinates; selective publication substitutes carrier robot coordinates. Both retain private branch states. Object-state errors are identical, with 16.74% mean reduction. Full-state increases free-state RMSE in three combinations and pusher RMSE in two; selective preserves both carrier errors. Full-state improves free-state RMSE in the other 24 combinations, so preservation does not establish accuracy dominance. Object and free-state aggregates mix position with velocity. The 27 combinations reuse three checkpoints. This publication test differs from the next-state recurrence ablation below and does not establish physical control benefit.

Shared-state versus isolated rollouts. A retrospective test uses 84 previously evaluated trajectories, with 12 per physical condition. Historical weights, context, observations, and actions are fixed across the rollout alternatives. The shared-state alternative retains separate recurrent memories but feeds the combined robot-object output into both next-step transitions. The three fits and three horizons reuse these trajectories. All 63 fit-condition-horizon comparisons give identical object-position and velocity RMSE for shared-state and isolated

rollouts. The isolated rollout exactly retains the standalone carrier robot prediction in every comparison; the shared-state alternative changes it in all 63. Robot diagnostic RMSE improves in 30 comparisons and worsens in 33 relative to shared-state recurrence. Isolation therefore controls reference preservation; it does not imply that the reference is more accurate. The archived closed-loop residual comparison separately reports endpoint differences after object adaptation.

VI. REAL-ROBOT CLOSED-LOOP PUSHING

We evaluate short-distance pushing with the five-DoF arm under intact and single-joint-lock conditions. The task requests a 20-pixel displacement along the task axis, approximately 35.1 mm under the recorded calibration. Each of the six conditions contributes three valid trials. All 18 core trials meet the criterion defined below; Table IV reports outcomes.

The controller uses visual object observations and joint feedback. It scores at least 10,000 candidate joint-reference sequences over a 50-step horizon, moves toward the selected reference at a 5-degree/s limit, and observes again before re-planning. Figure 4 illustrates recorded execution, while Fig. 5 shows the platforms.

TABLE IV
RECORDED CLOSED-LOOP PUSHING OUTCOMES. DOUBLE-LOCK RUNS
USE A SHORTER DISPLACEMENT AND INCLUDE CONFIGURATION
CHANGES; THEY ARE DIAGNOSTIC CASES RATHER THAN REPETITIONS OF
ONE FIXED CONTROLLER.

Condition	Goal (px)	Successes	Valid trials	Cycles
Intact	20	3	3	4
J1 lock	20	3	3	4–5
J2 lock	20	3	3	17
J3 lock	20	3	3	16–18
J4 lock	20	3	3	7
J5 lock	20	3	3	6–7
J1+J2	15	1	1	14
J2+J3	15	1	1	34
J3+J4	15	1	3	5–31

The start gate allows two pixels per image axis. Success requires an object-to-goal error within five pixels on each axis for three distinct consecutive observed frames. The archived endpoint statistic instead uses the coordinate-wise median of the last ten detected observations. Success is determined from the consecutive frames, not this median. Within each core condition, three repetitions reuse one preparation candidate archive, transformed online using the current joint state and remaining error. Archives differ across conditions.

Hardware candidates specify joint references q_t^{ref} . During prediction, a PD rule generates model actions as $a_t = \Pi_d(\text{clip}(5(q_t^{\text{ref}} - \hat{q}_t) - 0.5\hat{q}_t, -1, 1))$, using the returned robot state. Thus hardware candidate scoring includes state feedback within the rollout. With matched initial states and recurrent memories and deterministic branches, reference preservation also holds for matched reference sequences under this robot-state feedback rule: equal reference robot states produce equal actions at each step. The deployed robot block is independent of object state, so the same references yield identical robot predictions under shared-state and isolated recurrence.

Candidate references are joint offsets scaled by the remaining task-axis error, with locked references fixed at the diagnosed angles. Each observation initializes object position from the calibrated image measurement and object velocity to zero. The controller moves toward reference index 24, then observes again; it does not execute the preceding reference points. The archived strict simulation uses open-loop candidate execution, whereas the current large-scale comparison executes ten steps before replanning. Both differ from this hardware command.

All 23 formal hardware manifests record the same model configuration and the seed-27 selective checkpoint from `icra_confirmation_d3_query_selective_w10`. Its tensors match the seed-27 residual-training checkpoint used in the archived strict simulation. These are the archived deployment weights, distinct from the newly adapted fits.

Five valid double-lock trials request a shorter 15-pixel displacement, approximately 26.3 mm. J1+J2 and J2+J3 each succeed once; J3+J4 has two failures and one success. Candidate and controller settings changed across these diagnostic cases, so they are reported individually rather than pooled as fixed-

controller repetitions. Failed, invalid, and aborted attempts remain in the inventory. No matched hardware baseline was collected. The physical trials demonstrate execution, without establishing the physical validity of the archived simulation comparisons.

The archive also contains a same-weight candidate comparison: on 120 fresh D3 problems with 128 candidates and a 50-step open-loop execution, the deployed seed-27 checkpoint reduces terminal error by 8.72 mm relative to projected nominal dynamics (paired-problem 95% interval [6.48, 11.04] mm). Across seeds 7, 17, and 27, reductions relative to matched carriers are 2.42, 1.65, and 0.27 mm; only seed 7 has a positive interval. This comparison shares the suspect initialization and cannot establish physical benefit for the deployed weights.

VII. LIMITATIONS AND CONCLUSION

LockPusher limits the scope of object adaptation under diagnosed joint locks. Separate rollout states retain the intervention’s object prediction while preserving the specified robot forecast. Historical paired tests retain object-state error reductions without the observed collateral increases in carrier-relative robot error. Hardware trials demonstrate visual replanning with a configuration that satisfies preservation structurally; they do not isolate the benefit of separate rollouts. Simulation initialization defects leave physical control gains unverified.

The current evidence covers a fixed-orientation simulated box and short-distance physical pushing on one arm. Correct diagnosis is an input assumption, and preserving the carrier does not establish its accuracy. Extending this formulation to rotating objects and tasks that require adapting robot contact dynamics is a direction for further study.

ACKNOWLEDGMENTS

OpenAI Codex assisted with drafting and revising the abstract, introduction, related work, method, experiments, conclusion, and appendices; Chinese translation; figure and table code; and organization of the supplied experimental records.

APPENDIX A FIXED-CHECKPOINT PREDICTION ANALYSIS

These historical checkpoints differ from the newly adapted closed-loop models. For seed 47 under weak motors at horizon 50, position RMSE rises from 8.63 to 28.20 mm while velocity RMSE falls from 0.152 to 0.073 m/s. A mixed-unit aggregate obscures this component tradeoff.

The fixed-checkpoint study uses 8,400 independently reset trajectories under D3 locking, with 1,200 trajectories in each of seven physical conditions and a fixed-side contact-neighborhood policy. Three existing checkpoints are evaluated at horizons 10, 25, and 50. Across all 63 checkpoint–condition–horizon combinations, the selective wrapper changes no robot coordinates relative to its carrier, and maximum diagnosed-lock violation is zero. This tests the implementation of state isolation over extended rollouts.

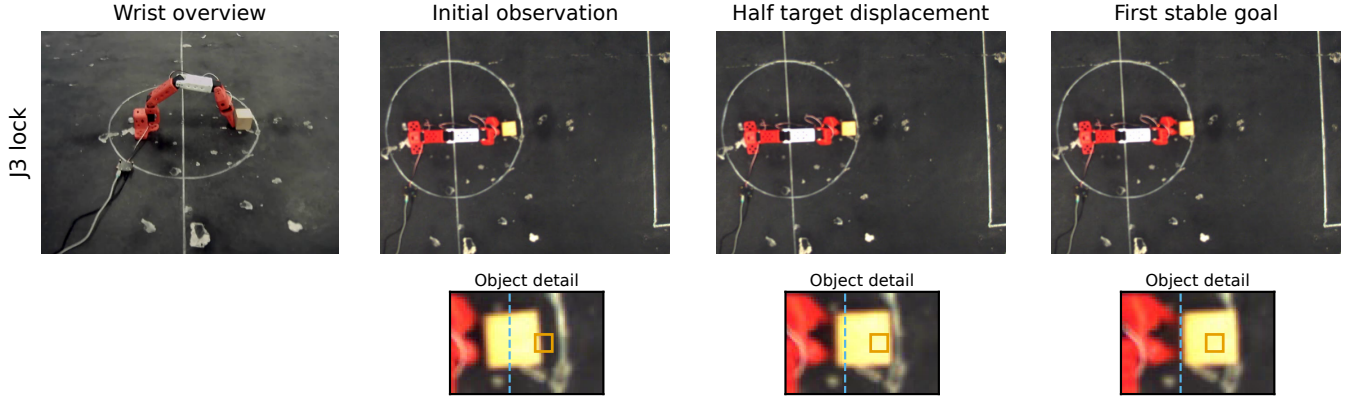


Fig. 4. J3-lock hardware execution. Overhead frames correspond to the initial observation, half the requested task-axis displacement, and the first three-frame goal streak. Each uses the latest camera capture preceding the monitor event. Insets show the recorded start coordinate (blue) and goal tolerance (orange). The wrist view shows the initial configuration.

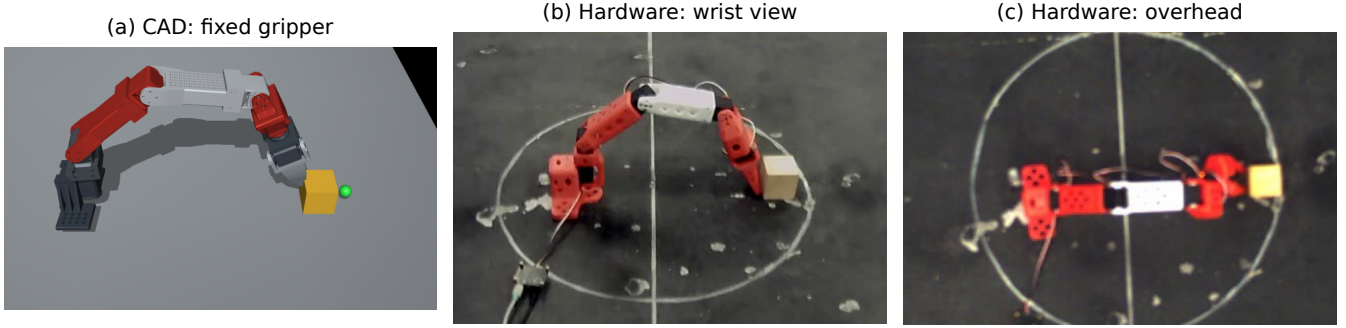


Fig. 5. Physical platform and CAD illustration. (a) Full-arm CAD rendering at an archived preparation configuration. (b,c) Wrist and overhead views from an archived J3 trial at different times. Quantitative simulations use a separate mechanism model with different geometry; the CAD panel illustrates the hardware configuration.

TABLE V

ADAPTATION BUDGET: ENDPOINT ERROR (MM) / SUCCESS (%). ALL THREE FITS PER BUDGET ARE SHOWN. EACH BUDGET USES 400 ADDITIONAL VALIDATION TRAJECTORIES; PRIOR SOURCE-MODEL TRAINING AND DEVELOPMENT DATA ARE EXCLUDED. C: NO OBJECT RESIDUAL; L: LOCKPUSHER; G: GLOBAL RESIDUAL.

Train	Fit	C	L	G
1,000	7	49.21/24.83	45.89/28.08	44.77/30.92
1,000	17	50.63/23.83	48.33/26.92	46.09/29.42
1,000	27	50.73/23.00	49.03/26.00	45.85/31.00
5,000	7	49.18/25.50	47.61/27.42	46.02/30.00
5,000	17	49.81/28.00	47.74/27.58	46.08/32.33
5,000	27	53.09/21.42	44.70/32.83	47.19/29.00
50,000	7	48.79/26.42	48.30/25.67	45.71/30.92
50,000	17	49.05/27.33	50.24/24.67	44.89/32.08
50,000	27	54.48/21.33	44.85/32.00	46.34/30.00

Figure 6 separates object position and velocity. The historical composite metric decreases in all 27 prespecified combinations for high damping, composition, and unseen composition, averaging 16.27%, but mixes position and velocity units.

A. Adaptation data budget

We vary the number of adaptation training trajectories while holding the number of optimization updates fixed. Nested subsets balanced across locked joints and physical profiles contain 1,000, 5,000, or 50,000 training trajectories, plus 400 validation trajectories at each budget. Three fits per method reuse the seed-27 source model’s trained robot and object dynamics; the added residual heads are initialized anew. Each receives 980 updates of batch size 512, using horizons 10 and 25 for 490 updates each. For each fit, we select the checkpoint with the lowest validation H25 object-position MSE, then evaluate it on 1,200 new shared problems under the same closed-loop protocol. These counts exclude all prior source-model training and development data.

Table V reports all fits. Averaged equally over the two lower budgets, LockPusher reduces endpoint error by 3.23 mm relative to removing its object residual. All six fit-budget contrasts improve, with paired 95% problem-bootstrap intervals excluding zero (2,000 resamples within the 40 lock-distance-profile strata, conditional on each fit). At 50,000 trajectories, two fits improve and one worsens. LockPusher’s mean endpoint error does not decrease monotonically with the training budget, and global adaptation has lower mean

endpoint error at each budget. These archived fits show lower endpoint error at the two lower budgets, without a monotonic scaling trend; physical benefit remains unverified.

APPENDIX B

MODEL IDENTITIES AND ADAPTATION DETAILS

Three sets of weights serve different experimental purposes. The large-scale adaptation study initializes every fit from the seed-27 primary model, reusing trained robot and object dynamics after removing its geometric residual and fits each variant with seeds 7, 17, 27, 37, 47, and 57. These seeds vary initialization of newly introduced residual parameters and minibatch order. They do not repeat pretraining. The archived deployment study retains its original weights; the physical controller uses the seed-27 selective checkpoint. The historical prediction replication uses the separate context-conditioned checkpoints with seeds 27, 37, and 47. Its context encoder and matched adapter are absent from the hardware input pipeline.

In the large-scale study, trainable parameters are restricted to the object head and the residual head appropriate to the variant. The robot backbone is frozen. The selective residual has 16 hidden units, while the global residual uses rank 10 and predicts all state coordinates. A no-residual model trains the object head without the geometric residual. Optimization uses Adam with learning rate 3×10^{-4} , minibatches of 512 trajectories, and gradient-norm clipping at 5. Each epoch visits the 50,000 training trajectories once in shuffled order and selects one contiguous window per trajectory without replacement at the trajectory level. A seeded random generator chooses the window start. The first five epochs use ten-step windows and the next five use 25-step windows. Validation runs after every epoch and also evaluates initialization; the selected checkpoint minimizes horizon-25 object-position MSE. Test trajectories are not loaded during fitting.

The earlier decision-trained checkpoints use a distinct predecessor procedure. Their object-stage loss combines one-step prediction with a weighted 25-step rollout loss and a candidate-ranking objective. These weights provide the starting point and archived deployment comparison. The newly fitted large-scale models use the normalized rollout objective in the main text rather than repeating that decision-training procedure.

APPENDIX C

PROJECTION AND REFERENCE PRESERVATION

The projection fixes locked position, velocity, and command coordinates at $(\bar{q}_j, 0, 0)$ and leaves free coordinates unchanged. Since the mask is binary, $\Pi_d \circ \Pi_d = \Pi_d$. Applying projection after every transition therefore enforces the diagnosed constraint at all finite prediction depths.

For preservation, compare the internal carrier to a standalone carrier with identical initial physical and recurrent states. Each receives the same projected action and diagnosis. Suppose their states agree at step t . Determinism of the transition implies equality at $t + 1$, including hidden states; the same projection preserves equality. Induction establishes

equality for the full candidate sequence. Selecting the carrier robot coordinates for publication then preserves the corresponding pusher coordinates through deterministic forward kinematics. The intervention’s object coordinates can change independently of this publication equality. The proof does not impose physical independence between the robot and object inside the intervention model.

A corrected published state must not become the input to the carrier recurrence. Otherwise, an object-to-robot dependency could change subsequent carrier outputs. The deployed robot block already excludes object-state input. The general two-branch construction also accommodates the historical coupled model, for which private state recurrence is consequential. Tests of those model families answer different implementation questions.

APPENDIX D

HISTORICAL CONTEXT AND ROUTING

Historical prediction experiments infer an eight-dimensional context from 25 observed transitions using a learned encoder. The matched carrier and intervention receive the same context and adapter. Centered modulation $g(z) = h(z) - h(0)$ recovers the zero-context modulation exactly. The historical routing variant selects intervention when $\|z\|_2 \geq 1.2$ and otherwise returns the carrier. This threshold was frozen after development with seed 27. The replication reports the direct selective comparison across every condition and identifies the three historical major conditions separately. Routing is a historical support heuristic and is not used to identify a lock or to process observations in the physical controller.

The replication uses fixed D3 diagnosis, seven physical conditions, and 1,200 newly reset trajectories per condition. It reuses three historical checkpoints without retraining. Each trajectory has 150 actions, and terminal errors are evaluated at horizons 10, 25, and 50. The historical composite object-state RMSE mixes position and velocity units. Position and velocity errors are therefore retained separately in the accompanying complete results. Windows, model variants, and checkpoint repetitions do not multiply the count of 8,400 trajectories.

APPENDIX E

SIMULATION AND HARDWARE EXECUTION

The large-scale simulation uses 50-step trajectories at 5 ms per step. Each of five lock conditions contributes 10,000 training, 400 validation, and 1,200 test trajectories with disjoint reset identifiers. Closed-loop planning uses 1,200 independently generated problems, with 120 problems for each lock and distance-band combination. Within a repetition, candidate randomization and initial problems are paired across methods. After models select different commands, each method continues from its own resulting state. Each cycle predicts 50 steps, executes ten, and replans from a new observation, for five cycles.

Hardware execution uses visual object position and joint feedback. Object velocity is initialized to zero at each observation. Candidate references are transformed using measured

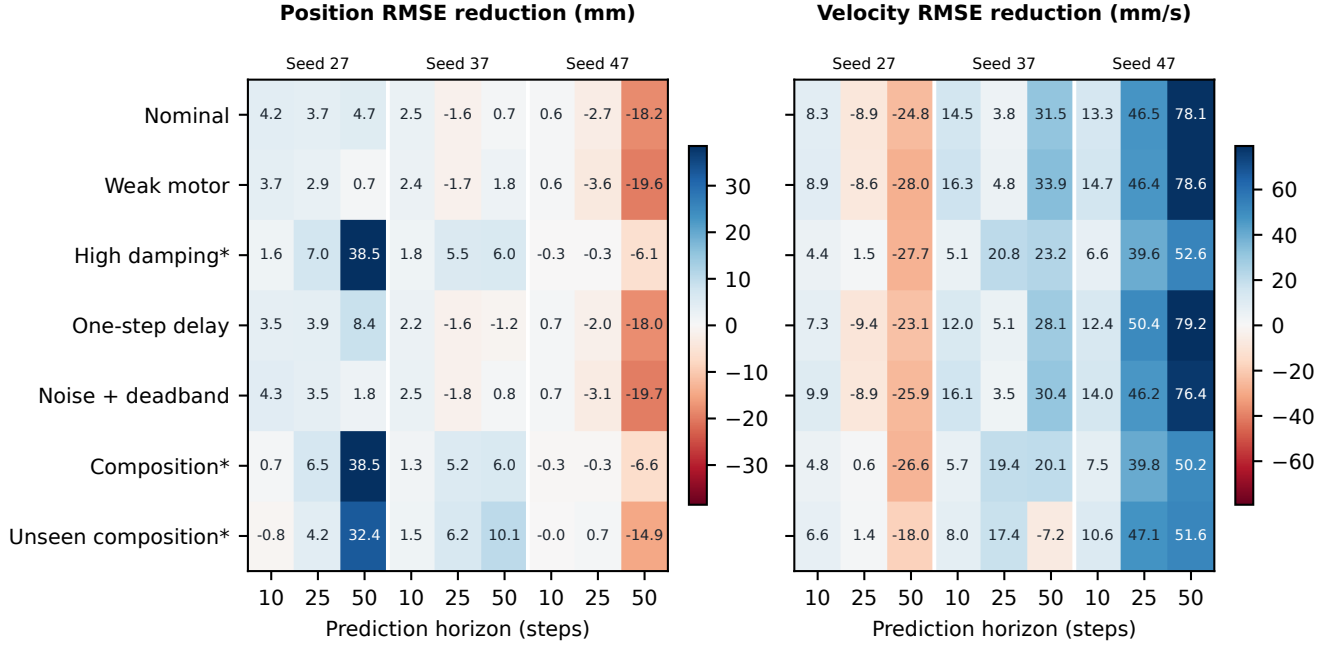


Fig. 6. All 63 historical-model comparisons. Values are carrier RMSE minus adapted RMSE: positive means improvement, negative means degradation. Position (mm) and velocity (mm/s) use separate color scales. Seeds 27, 37, and 47 identify historical fits, not training epochs. Stars mark the historical prespecified conditions.

joints and remaining task-axis error. The controller moves at the recorded rate limit toward reference index 24 and then observes again. This operation does not execute the preceding 24 reference points. The 18 core trials and five double-lock diagnostic trials retain their original controller settings and outcome criteria in the hardware results. Simulation execution lengths and hardware reference indices are different protocol quantities.

APPENDIX F REPRODUCTION AND AGGREGATION

The adaptation implementation is `scripts/ipwm_scale_train.py`; its training protocol records the window schedule, optimizer, common initialization, and validation rule. The scale dataset audit records split counts and reset disjointness. Complete results and comparison intervals are generated from the corresponding stored trajectories and planning outcomes. The historical replication uses its own protocol and checkpoint manifest. Source paths preserve the original IPWM identifiers although the manuscript names the method LockPusher.

Prediction RMSE first averages squared coordinate errors, then takes the square root. Position and velocity have separate units. Planning terminal error is the Euclidean object-to-goal distance. Each adaptation fit in the main closed-loop table uses one candidate-sampling repeat; its interval resamples paired problems within that repeat. For the aggregate calibration interval, paired differences are averaged across six fits at each shared problem before stratified resampling within lock and distance cells. Variation across newly adapted fits is

reported separately from repetitions of a fixed deployment checkpoint. CPU and MuJoCo Warp transition comparisons check numerical consistency of the simulator acceleration; they do not establish a learned-model accuracy or control advantage.

The scale-data contact reference is (0.34396525, 0.79659639, 0.79649504, 1.42124113, 1.55083163) rad. Data generation adds independent Gaussian joint-position perturbations with standard deviation 0.015 rad and joint-velocity perturbations with standard deviation 0.01 rad/s, with locked velocity set to zero. Both object translation coordinates are perturbed uniformly within $[-4, 4]$ mm. A solver equality constrains the locked position to its reference angle.

APPENDIX G SOURCE-MODEL TRAINING RECIPE

The source model combines separately initialized robot and object weights. The robot path starts from a shared graph baseline trained for 240 epochs, transfers its robot tensors, and undergoes 40 robot-only epochs in the strict-pusher stage. The primary stage copies this robot predictor, zeros the two topology-input columns, initializes its object head from the shared baseline, and selects object epoch 40. Its recorded ordinary-data recipe contains 40 training and 15 validation trajectories, each with 150 transitions; the decision stage records 320 training and 80 validation groups, with 32 candidates sharing an initial state in each group. Repeated rollout windows and candidates are not independent collected trajectories. The retained source summaries do not bind the

exact decision-training archive or establish a complete duplicated development-data total. Additional adaptation budgets exclude this prior training and development.

APPENDIX H HISTORICAL CANDIDATE-SELECTION PROTOCOL

The strict, D3 post-freeze, and oracle comparisons share the suspect initial-pose construction. Their numerical results below do not establish physical pushing performance. The other archives retain their separate, unresolved physical-validity and provenance boundaries.

This appendix distinguishes the historical strict candidate-selection study (model seeds 7/17/27), the coupled-model routing and isolation study (fixed checkpoints 27/37/47), and the calibrated transfer study (completed fits 107/117/127). Repeated windows, candidate commands, and evaluations of one checkpoint are not additional training seeds or independent trajectories. Their archived metric definitions and negative results are retained; none are pooled with the current six-fit adaptation experiment.

The historical strict candidate-selection study uses MuJoCo planar pushing with the five-joint simulation model `arm_push.xml` and a four-dimensional planar object state. The calibrated `genkiarm_push.xml` transfer study is reported separately. D2 and D4 locks form the frozen development distribution. Seeds 7/17/27 use separate model initializations and candidate archives. Formal evaluation contains 400 matched groups per seed (80 episodes with five stages each), 128 action sequences per group, and 50 simulator steps per sequence. Methods share the current state, goal, diagnosis, and action candidates. Simulator contact, minimum geometry distance, and realized terminal state are shared evaluation labels, not prediction inputs. Each model selects by predicted terminal cost; only the oracle selects by realized cost.

Prediction training is configured with 20 domains, two 150-step trajectories per domain: 40 trajectories and 6,000 transitions per seed. The decision-training stage uses 320 unique candidate groups per model seed and 80 validation groups, with 32 candidates per group. These counts describe residual decision training rather than the data used to train predecessor checkpoints. The 128-candidate evaluation groups are reported separately from these training groups.

The formal rows are nominal WM, analytic projection/carrier, same-capacity global residual, full-state residual, selective publication, decision weight zero, decision weight 10, and an oracle that selects by realized terminal cost. The oracle measures headroom and is never presented as a deployable baseline. Metrics follow the causal order of use: maximum diagnosed-lock violation; predicted contact-manifold proxy and realized minimum distance; physical contact; contact-candidate response RMSE; Spearman/Kendall and top-1 regret; and realized selected-candidate terminal error/success. The last stage is open-loop candidate selection, not closed-loop MPC.

D3 was excluded from gradient fitting and epoch selection, but earlier versions of the project inspected D3 diagnostics.

TABLE VI
STRICT THREE-SEED RESULTS VERSUS THE NOMINAL WORLD MODEL.
POSITIVE REGRET/ERROR ENTRIES DENOTE REDUCTIONS; SUCCESS IS
PERCENTAGE POINTS.

Metric	Global	LockPusher
Response RMSE reduction	−270.04%	−247.90%
Spearman change	+0.0375	+0.0378
Top-1 regret reduction	+19.76%	+18.28%
Terminal error reduction	+4.04%	+3.73%
Success change (pp)	+1.58	+1.17

We therefore call it held out from fitting, not untouched confirmation. Historical simplified-arm and calibrated GenkiArm spectra are retained below as disclosed development/failure evidence; they are not pooled with the historical strict table.

APPENDIX I HISTORICAL RESULTS

The current adaptation experiment is reported in the main experimental section. Figure 6 gives the separate 8,400-trajectory replication with all 63 component-wise prediction comparisons. The studies below retain their original checkpoints, query sets, and aggregation units.

A. Strict six-stage fault-adaptation result

Table VI reports the matched D2/D4 candidate-selection comparison under the protocol above. Its task outcomes are realized open-loop executions of the selected candidates. Response RMSE here is the root mean squared Euclidean XY prediction error on contact candidates, whereas the main prediction analysis averages squared errors over coordinates before taking the root.

The global residual reduces top-1 regret and terminal candidate error in 3/3 seeds. The matched comparison separates this adaptation gain from selective publication. Relative to the same-capacity global residual, selective LockPusher increases mean regret by 2.07% and terminal error by 0.32%, decreases success by 0.42 percentage points, and increases Spearman by 0.00037. Full-state and selective publication give identical outputs in all three seeds. Here the robot transition does not depend on object state, and the carrier shares its robot parameters with the intervention model. Object corrections cannot change either robot trajectory, making selective and full-state publication equivalent in this configuration.

The realized-cost oracle has mean terminal error 0.03758 m versus 0.04675 m for nominal selection, leaving 9.17 mm or 19.63% mean endpoint headroom (positive in 3/3 seeds). The oracle is privileged and non-deployable; it shows that candidate quality is not exhausted, while the global residual closes only a fraction of the available outcome gap.

We additionally registered a post-freeze D3 candidate-query test before generating its seed-91031 archive (200 groups from 40 episodes with five stages each, 128 candidates, 50 steps). D3 had been inspected in historical exploration, so this is not presented as pristine domain confirmation. Global

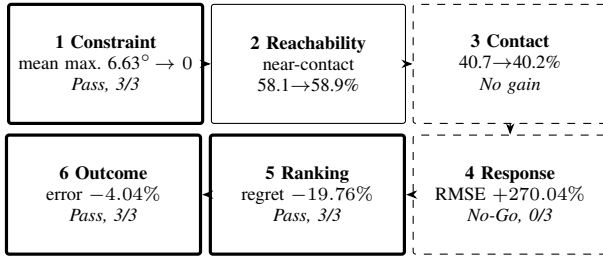


Fig. 7. Six-stage evidence chain. Stage 1 reports the projection ablation (mean of three seed maxima); stages 2–6 compare the same-capacity global residual with the nominal world model. Solid boxes pass a frozen direction/constraint gate; dashed boxes expose a failed transfer. “Outcome” is the realized selected open-loop candidate, not receding-horizon MPC.

TABLE VII

ANALYTIC-PROJECTION REMOVAL ABLATION OVER THREE SEEDS. THE DEGREE ROW AVERAGES THE THREE SEED-LEVEL MAXIMUM POSITION VIOLATIONS.

Constraint metric	No projection	Projection	Seeds
Max position drift (rad)	0.077–0.153	0	3/3
Mean seed maximum (deg)	6.63	0	3/3
Max velocity violation (rad/s)	0.384–0.660	0	3/3

residual versus nominal reduces regret by only 9.77% (2/3 seeds) and terminal error by 2.00% (2/3), while success rises 2.17 points (3/3); response RMSE worsens 263.63% and Spearman changes by -0.0148 . Selective LockPusher again fails matched attribution: relative to global, regret increases by 3.82%, terminal error increases by 0.50%, and success decreases by 0.83 percentage points. Thus the fresh-query result confirms no large transferable task-performance advantage and bounds the D2/D4 development claim.

The response column exposes the central diagnostic result. Both decision-trained models choose better actions while their contact-candidate response RMSE becomes substantially worse. Consequently, neither one-step nor rollout RMSE can stand in for action-ranking evidence in fault-aware contact planning.

Table VII isolates a different contribution. Removing projection permits large diagnosed-joint drift, whereas analytic projection sets both violations to exactly zero. Candidate regret and terminal outcome change little, so projection is claimed as a structural correctness guarantee, not as a success-rate improvement.

B. Decision-loss and publication ablations

With the same architecture and 40-epoch protocol, weight-10 counterfactual decision training versus weight zero improves top-1 regret by 5.98% and terminal error by 1.40% on average, but only in 2/3 seeds; it decreases success by 0.50 points and worsens response RMSE by 356.44%. Conversely, weight zero improves response RMSE by 24.84% and success by 1.67 points over nominal in 3/3 seeds. The paired objective therefore shifts the model along a prediction–decision tradeoff; it is not a uniformly improving component.

TABLE VIII

HISTORICAL CALIBRATED-GENKIARM COMPOSITE OBJECT-STATE RMSE IMPROVEMENT (%) OVER THE MECHANISM-MATCHED CARRIER; RETAINED AS A NEGATIVE TRANSFER BOUNDARY.

Method	107	117	127	Mean
Routed selective	0.604	-0.784	0.718	0.179
Raw selective	1.420	-1.913	2.546	0.684

C. Retrospective calibrated-GenkiArm boundary

The archived object metric in this and the following historical prediction tables combines position (m) and velocity (m/s). A positive composite improvement does not imply improvement of both components.

The earlier 27/37/47 checkpoint matrix below was generated on the simplified `arm_push.xml` development model. It is retained to explain the origin of state isolation, but it is not the paper’s calibrated-model performance evidence. The retrospective audit used `genkiarm_push.xml`, disjoint query seeds, and fresh training seeds 107/117/127.

For routed selective LockPusher, only 2/3 seeds are positive and the hierarchical paired-bootstrap 95% interval is $[-0.745\%, 0.797\%]$. The raw selective interval is $[-1.870\%, 2.502\%]$. Both fail the preregistered +5% mean, 80% positive-seed, and positive-lower-bound gates; three of five planned seeds completed. Bootstrap resamples seeds and paired windows within condition/horizon, without modeling temporal dependence. Free-state regression and locked-coordinate violation are exactly zero in all three seeds. The retrospective decision is therefore an inconclusive performance comparison with support for the measured isolation property. These rows are not pooled with the historical strict result.

D. Historical simplified-model failure

On seed 7, D3 mixed-unseen, H50, the original intervention has composite free-state RMSE 0.808 versus 0.762 for the shared projected baseline, a 6.01% regression. The paired MSE-difference interval is $[0.06885, 0.07538]$, and 93.3% of terminal windows regress. Composite object-state RMSE improves 36.49% and lock violation remains zero. The free-state score mixes joint position and velocity units. The failure therefore exposes a real tradeoff: object correction feeds back into unaffected robot prediction even though feasibility is projected.

The post-design query set makes the repair comparison direct (Table IX). Full-state and isolated methods have exactly the same composite object-state RMSE in every cell because the intervention keeps its private physical trajectory. Full-state rollout nevertheless regresses free-joint RMSE in 3/27 cells, with a 6.20% worst case. Isolation removes those changes without changing object prediction.

E. State isolation removes collateral regression

Table X reports the post-design query-set cells. Composite object-state RMSE improves in 27/27 cells. The smallest gain is 1.58% (checkpoint seed 37, mixed unseen, H50),

TABLE IX

ISOLATION ABLATION ON THE POST-DESIGN QUERY SET. COMPOSITE OBJECT-STATE GAIN IS THE MEAN OVER CHECKPOINT-SEED MEANS; FREE REGRESSION IS THE WORST CELL.

Method	Object gain	Free regr.	Cells
Carrier	0	0	0/27
Full-state	16.74%	6.20%	3/27
LockPusher	16.74%	0	0/27

TABLE X

COMPOSITE OBJECT-STATE RMSE IMPROVEMENT (%) OVER THE MECHANISM-MATCHED CARRIER IN ACTIVE PHYSICAL-OOD ROUTES.

Seed	Condition	H10	H25	H50
27	high damping	7.05	3.06	7.58
27	composition	7.70	2.03	19.54
27	mixed unseen	9.08	3.26	3.43
37	high damping	7.31	21.64	37.41
37	composition	8.52	20.93	30.23
37	mixed unseen	10.39	21.00	1.58
47	high damping	8.57	34.68	33.00
47	composition	10.63	35.53	30.52
47	mixed unseen	12.53	38.86	25.95

and the largest is 38.86%. Mean improvement over the three checkpoint-seed means is 16.74%, with descriptive bootstrap interval [6.97%, 25.59%]. The original seed-specific query sets independently retain 27/27 positive cells, 1.34–39.97% range, and 17.04% mean. Across all cells, free-joint and pusher changes relative to the carrier are numerically exactly zero, as predicted by Eq. (5); lock violation RMSE is zero.

The four low-context conditions route to the carrier and therefore tie it exactly. This is intentionally conservative: an earlier topology-only router improved severe physical shifts but increased composite object-state RMSE over the shared projected baseline by up to 32.73% in nominal-like H25/H50 cells. A support selector using a 20-step context prefix and five-step validation suffix rejected intervention in all seven tested conditions. These negative controls motivate physical-context routing but do not establish that norm thresholding is optimal.

F. Routing margins and query replication

Table XII reports context-norm ranges across the three checkpoints on the original physics spectrum. Nominal, noisy-deadband, and weak-motor conditions have wide margins below 1.2. Delay is closer (maximum 1.12), and mixed unseen is the closest active condition (minimum 1.26). Thus the observed decisions are stable on these checkpoints, although the 0.06 mixed-unseen margin is too small to call the router calibrated under arbitrary shift.

The post-design query set changes both calibration and evaluation trajectories while leaving weights, K, scale, threshold, and domains frozen. Its 16.74% mean is close to the original 17.04%; both have 27/27 positive active cells. This replication argues against memorization of one action/initialization set. It does not test a new object geometry, simulator, or training seed, and is labelled accordingly throughout.

TABLE XI

AGGREGATE PREDICTION AUDIT. “CHANGE” IS RELATIVE TO CARRIER.

Statistic	Result
Positive object cells	27/27
Object improvement range	1.58–38.86%
Mean over checkpoint-seed means	16.74%
Seed-bootstrap 95% interval	[6.97, 25.59]%
Maximum free-joint change	0
Maximum pusher change	0
Maximum lock violation RMSE	0

TABLE XII

CONTEXT NORM RANGE AND FROZEN ROUTE ACROSS CHECKPOINT SEEDS.

Condition	Minimum	Maximum	Route
Nominal	0.49	0.52	carrier
Noisy deadband	0.48	0.52	carrier
Weak motor	0.77	0.83	carrier
Delay 1	1.07	1.12	carrier
High damping	1.33	1.54	intervention
Composition	1.45	1.50	intervention
Mixed unseen	1.26	1.45	intervention

G. Prediction gain does not imply control gain

We evaluated a frozen guarded CEM planner using 60 approach and 90 push steps, horizon 3, eight candidates, one CEM update, and a fixed 0.85 action-deviation guard. The prior 50 mm success tolerance is invalid for this target split: targets start only 30–40 mm from the object, so zero-contact episodes count as success. We instead report terminal distance, contact, and displacement.

All audited episodes make contact, but LockPusher improves on carrier MPC for only one of three seeds. This historical controller study does not establish a closed-loop improvement. Its three-target protocol is distinct from the 1,200-problem adaptation study in the main text. This result is consistent with the method’s guarantee: isolation controls which state is published, not how a finite-sample optimizer converts prediction differences into actions.

H. Reproducibility

Frozen YAML files define domain splits, context scaling, adapters, and routing. Per-window squared errors regenerate Table XI and its bootstrap interval; row counts are detailed below. Unit tests verify exact projection, carrier publication, and preservation of private intervention feedback with a robot-coupled toy model. Failed gates and protocol corrections are retained in the repository rather than overwritten.

I. Threats to validity

Post-hoc structure. State isolation was invented after the original free-state failure; only query seed 57 is post-design. *Dependence.* The 27 cells share three checkpoints and three domains, so they are not treated as 27 independent replicates. *Baseline scope.* Carrier matching isolates the intervention effect, but comparisons to AdaptiGraph or PIN-WM are concep-

TABLE XIII
D3 HIGH-DAMPING MEAN TERMINAL DISTANCE (MM) OVER THREE
HELD-OUT TARGETS. LOWER IS BETTER.

Seed	Nominal IK	Carrier MPC	LockPusher MPC
27	16.99	13.75	19.45
37	16.99	18.77	15.67
47	16.99	34.25	39.11

TABLE XIV
OBSERVED FAILURES AND THEIR IMPLICATIONS. RELATIVE CHANGES
USE THE COMPARISON NAMED IN EACH ROW OR THE NOMINAL MODEL
FOR THE GLOBAL RESIDUAL.

Comparison	Observation	Implication
Hard mask only	contact not established	separate feasibility/contact
No analytic projection	mean seed max. 6.63 deg	retain hard projection
Decision weight 10 vs. 0	RMSE rises 356.44%	prediction/ranking tradeoff
Global residual	regret falls 19.76%, 3/3	adaptation improves ranking
Selective vs. global	regret rises 2.07%	no selective advantage
Historical guarded MPC	improves 1/3 seeds	no consistent control gain

tual rather than reproduced. *Metric scope.* The archived composite RMSE mixes position and velocity units and weights the four coordinates uniformly and does not imply useful action ranking. The No-Go MPC result makes this boundary visible. *Router scope.* Context norm is an engineering support heuristic, not a calibrated probability of harm. *Simulation scope.* The shared variable-node transition Gate improves both GenkiArm and Panda in 2/3 seeds, but the downstream object/contact criterion requiring improvement on both robots is met in none of the three seeds. A no-weld scripted Panda grasp succeeds in 5/5 small pose perturbations, and both simulated fixed eye-to-hand views retain object visibility under the frozen extrinsic perturbations; these are task/observability feasibility, not learned Grasp, RGB prediction, or contact-transfer evidence.

APPENDIX J FAILURE ANALYSIS

Table XIV connects the observed failures to the corresponding model or evaluation choices. Constraint satisfaction, response accuracy, and action selection impose separate requirements: exact projection can coexist with inaccurate contact predictions, and lower candidate regret can coexist with higher response RMSE.

Lock violation measures diagnosed constraint satisfaction. Reachability and physical contact determine whether a candidate can interact with the object; paired ranking and realized outcomes then measure selection quality. The experiments support exact lock constraints and adaptation gains in candidate regret, but not improved response RMSE or an additional benefit from selective publication. Historical receding-horizon control improves in only one seed.

A. Artifact-to-claim mapping

Three artifact types support these historical studies: raw trajectory windows support object statistics, model outputs support isolation equality, and environment logs support contact

and terminal-distance checks. Aggregate tables are regenerated rather than hand selected. The code path accepts an explicit query seed so that checkpoint initialization and evaluation queries can be varied separately. Hardware outcomes in Table IV are reported separately from simulator measurements.

APPENDIX K EVALUATION AND AUDIT DETAILS

A. Historical candidate groups and aggregation

Each seed contributes 400 groups and each group contains 128 candidates sharing one initial state, goal, and diagnosis. Spearman and Kendall correlations are computed within groups and averaged over groups. Top-1 regret is the realized cost of the model-selected candidate minus the minimum realized cost in that same group. Terminal error and success are evaluated on the selected physical candidate, never on its prediction. Seed-level relative changes are computed first and then averaged; the three seeds, rather than 153,600 candidates, are the independent units used for direction counts. Continuous minimum geometry distance and simulator contact are stored separately so that near-contact is not relabeled as contact.

B. Window construction and aggregation

Each test trajectory contains 150 actions and 151 states. For horizon H , autonomous prediction starts at indices separated by H and rolls for exactly H actions; only the terminal prediction enters that window’s statistic. Squared error is computed per trajectory/window, averaged, and square-rooted to obtain RMSE. Free-joint error masks position and velocity of the diagnosed coordinate. Object error averages four planar coordinates, mixing position (m) and velocity (m/s) without a common physical unit. These archived composite values are not position RMSE; component-wise results for the independent replication appear in Fig. 6. Pusher error applies the same forward kinematics to predicted and true joint positions.

The original spectrum audit stores 5,040 per-window/method rows per checkpoint. The post-design audit restricts generation to three active regimes and, after adding the full-state ablation, stores 2,880 rows per checkpoint. Carrier, full-state, isolated, and routed predictions use identical initial states and actions. Exact object equality between full-state and isolated rollouts is checked cell by cell rather than inferred from their aggregate.

C. Uncertainty and multiplicity

The bootstrap treats the mean across nine active cells within a checkpoint as one observation and resamples three checkpoint means. Correlated horizons and domains are therefore not presented as 27 independent seeds. With only three observations, the interval is a stability summary, not precise population inference. We consequently report every cell and the positive-cell count; no null-hypothesis threshold accepts a population-level claim.

The seed-7 diagnostic interval has a different purpose: it resamples 30 trajectory-level means of paired terminal-window

MSE differences within one failure cell. It shows the free-state regression is systematic within that query set, not that it generalizes to a population of training seeds. Keeping the two analyses separate prevents trajectory count from inflating the effective number of model initializations.

D. Controller audit protocol

The historical controller comparison shares approach/push references, damage masks, action limits, initial object position, and held-out targets. Its CEM settings and corrected terminal-distance metric are given above. An earlier 30-step approach and 40-step push failed to establish contact, motivating the reported 60+90-step protocol. Contact and object displacement are recorded alongside terminal distance.

E. Reproduction boundary

The evaluator records checkpoint and query seeds separately. Reproducing the historical candidate-selection table requires the three strict checkpoints, V2 candidate archives for seeds 7/17/27, and the fixed 400-by-128-by-50 evaluator. The formal checkpoint loader rejects missing, unexpected, or shape-mismatched tensors. Separate summarizers regenerate the historical strict result, decision-loss comparison, same-capacity global control, and projection-removal ablation from JSON without loading weights. Tests cover pairing, held-out-lock exclusion, the soft-regret objective, exact projection, and private/published state recurrence.

Training depends on staged predecessor checkpoints, while evaluation is scripted from the selected weights and candidate archives. Reproducing training therefore requires the predecessor runs as well as the final residual-training configuration.

REFERENCES

- [1] C. Finn and S. Levine, “Deep visual foresight for planning robot motion,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 2786–2793.
- [2] M. Bauza, F. R. Hogan, and A. Rodriguez, “A data-efficient approach to precise and controlled pushing,” in *Proceedings of the 2nd Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 87, 2018, pp. 336–345.
- [3] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, “Robots that can adapt like animals,” *Nature*, vol. 521, pp. 503–507, 2015.
- [4] F. Yang, C. Yang, D. Guo, H. Liu, and F. Sun, “Fault-aware robust control via adversarial reinforcement learning,” in *2021 IEEE 11th Annual International Conference on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER)*, 2021, pp. 109–115.
- [5] S. Kim, B. Lim, Y. Lee, and F. C. Park, “SE(2)-equivariant pushing dynamics models for tabletop object manipulations,” in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 427–436. [Online]. Available: <https://proceedings.mlr.press/v205/kim23b.html>
- [6] Z. Zhong, S. Golestaneh, and C. Chamzas, “ActivePusher: Active learning and planning with residual physics for nonprehensile manipulation,” in *IEEE International Conference on Robotics and Automation*, 2026. [Online]. Available: <https://arxiv.org/abs/2506.04646v4>
- [7] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Proceedings of Robotics: Science and Systems*, 2021.
- [8] A. Allevato, E. S. Short, M. Pryor, and A. Thomaz, “TuneNet: One-shot residual tuning for system identification and sim-to-real robot task transfer,” in *Proceedings of the Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 100, 2020, pp. 445–455. [Online]. Available: <https://proceedings.mlr.press/v100/allevato20a.html>
- [9] K. Zhang, B. Li, K. Hauser, and Y. Li, “AdaptiGraph: Material-adaptive graph-based neural dynamics for robotic manipulation,” in *Robotics: Science and Systems*, 2024.
- [10] W. Li, H. Zhao, Z. Yu, Y. Du, Q. Zou, R. Hu, and K. Xu, “PIN-WM: Learning Physics-INformed world models for non-prehensile manipulation,” in *Robotics: Science and Systems*, 2025.